

Chapter 10: Pointers

Pointers are often thought to be the most difficult aspect of C. It's true that many people have various problems with pointers, and that many programs founder on pointer-related bugs. Actually, though, many of the problems are not so much with the pointers *per se* but rather with the memory they point to, and more specifically, when there *isn't* any valid memory which they point to. As long as you're careful to ensure that the pointers in your programs always point to valid memory, pointers can be useful, powerful, and relatively trouble-free tools. (We'll talk about memory allocation in the next chapter.)

[This chapter is the only one in this series that contains any graphics. If you are using a text-only browser, there are a few figures you won't be able to see.]

A pointer is a variable that points at, or refers to, another variable. That is, if we have a pointer variable of type `pointer to int`, it might point to the `int` variable `i`, or to the third cell of the `int` array `a`. Given a pointer variable, we can ask questions like, "What's the value of the variable that this pointer points to?"

Why would we want to have a variable that refers to another variable? Why not just use that other variable directly? The answer is that a level of *indirection* can be very useful. (*Indirection* is just another word for the situation when one variable refers to another.)

Imagine a club which elects new officers each year. In its clubroom, it might have a set of mailboxes for each member, along with special mailboxes for the president, secretary, and treasurer. The bank doesn't mail statements to the treasurer under the treasurer's name; it mails them to "treasurer," and the statements go to the mailbox marked "treasurer." This way, the bank doesn't have to change the mailing address it uses every year. The mailboxes labeled "president," "treasurer," and "secretary" are a little bit like pointers--they don't refer to people directly.

If we make the analogy that a mailbox holding letters is like a variable holding numbers, then mailboxes for the president, secretary, and treasurer aren't quite like pointers, because they're still mailboxes which in principle could hold letters directly. But suppose that mail is never actually put in those three mailboxes: suppose each of the officers' mailboxes contains a little marker listing the name of the member currently holding that office. When you're sorting mail, and you have a letter for the treasurer, you first go to the treasurer's mailbox, but rather than putting the letter there, you read the name on the marker there, and put the mail in the mailbox for that person. Similarly, if the club is poorly organized, and the treasurer stops doing his job, and you're the president, and one day you get a call from the bank saying that the club's account is in arrears and the treasurer hasn't done anything about it and asking if you, the president, can look into it; and if the club is so poorly organized that you've forgotten who the treasurer is, you can go to the treasurer's mailbox, read the name on the marker there, and go to *that* mailbox (which is probably overflowing) to find all the treasury-related mail.

We could say that the markers in the mailboxes for the president, secretary, and treasurer were *pointers* to other mailboxes. In an analogous way, pointer variables in C contain pointers to other variables or memory locations.

[10.1 Basic Pointer Operations](#)

[10.2 Pointers and Arrays; Pointer Arithmetic](#)

[10.3 Pointer Subtraction and Comparison](#)

[10.4 Null Pointers](#)

[10.5 ``Equivalence" between Pointers and Arrays](#)

[10.6 Arrays and Pointers as Function Arguments](#)

[10.7 Strings](#)

[10.8 Example: Breaking a Line into ``Words"](#)

Read sequentially: [prev](#) [next](#) [up](#) [top](#)

This page by [Steve Summit](#) // [Copyright](#) 1995, 1996 // [mail](#) [feedback](#)